

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 5

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

*I **M.S.JASWANTH(192525093)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations.BTL: BL3 (Apply)

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types.BTL: BL3 (Apply)

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories.BTL: BL4 (Analyze)

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions.BTL: BL3 (Apply)

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version.BTL: BL3 (Apply)

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior.BTL: BL4 (Analyze)

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor.

Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display.BTL: BL3 (Apply)

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation.BTL: BL3 (Apply)

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification.BTL: BL3 (Apply)

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations.BTL: BL4 (Analyze)

Question 1: Library Book Management using Classes & Encapsulation

Program

```
import java.util.Scanner;

class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    Book(int id, String t, String a, double p) {
        bookId = id;
        title = t;
        author = a;
        price = p;
        issued = false;
    }

    public int getBookId() {
        return bookId;
    }

    public String getTitle() {
        return title;
    }

    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued Successfully");
        } else {
            System.out.println("Book Already Issued");
        }
    }
}
```

```

    }

    public void returnBook() {
        if (issued) {
            issued = false;

            System.out.println("Book Returned Successfully");
        } else {
            System.out.println("Book Already Available");
        }
    }

    public void display() {
        System.out.println(bookId + " " + title + " " + author + " " + price + " Status: " + (issued ? "Issued"
: "Available"));
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[3];

        books[0] = new Book(101, "Java", "James", 450);
        books[1] = new Book(102, "Python", "Guido", 500);
        books[2] = new Book(103, "C++", "Bjarne", 600);

        int choice;

        do {
            System.out.println("\n1.Display");
            System.out.println("2.Issue");
            System.out.println("3.Return");
            System.out.println("4.Search");
            System.out.println("5.Exit");

            choice = sc.nextInt();

            switch(choice){
                case 1:
                    for(Book b:books)
                        b.display();

                    break;
                case 2:
                    System.out.print("Enter Book ID:");

                    int id=sc.nextInt();

```

```

        for(Book b:books)
            if(b.getBookId()==id)
                b.issueBook();

        break;
case 3:
    System.out.print("Enter Book ID:");
    id=sc.nextInt();
    for(Book b:books)
        if(b.getBookId()==id)
            b.returnBook();

    break;
case 4:
    System.out.print("Enter Book ID:");
    id=sc.nextInt();
    boolean found=false;
    for(Book b:books){
        if(b.getBookId()==id){
            b.display();
            found=true;
        }
    }
    if(!found)
        System.out.println("Book Not Found");
    break;
    }
}while(choice!=5);
}
}

```

Sample Output

1.Display

101 Java James 450.0 Status: Available

102 Python Guido 500.0 Status: Available

103 C++ Bjarne 600.0 Status: Available

Enter Book ID:101

Book Issued Successfully

Test Cases

Input	Expected Output
Issue Book 101	Book Issued Successfully
Issue Book 101 again	Book Already Issued
Return Book 101	Book Returned Successfully
Search 102	Displays Python Book
Search 999	Book Not Found

Question 2: Employee Payroll System using Inheritance

Program

```

class Employee {
    String name;
    Employee(String n){
        name=n;
    }
    double calculateSalary(){
        return 0;
    }
}

class PermanentEmployee extends Employee{
    double basic;
    PermanentEmployee(String n,double b){
        super(n);
        basic=b;
    }
    double calculateSalary(){
        return basic+5000;
    }
}

class ContractEmployee extends Employee{
    double hours;
    double rate;
    ContractEmployee(String n,double h,double r){
        super(n);
        hours=h;

```

```

        rate=r;
    }

    double calculateSalary(){

        return hours*rate;
    }
}

public class Main{

    public static void main(String args[]){

        Employee e;

        e=new PermanentEmployee("Arun",30000);

        System.out.println("Permanent Salary = "+e.calculateSalary());

        e=new ContractEmployee("Kiran",120,250);

        System.out.println("Contract Salary = "+e.calculateSalary());

    }

}

```

Output

Permanent Salary = 35000.0

Contract Salary = 30000.0

Test Cases

Employee	Expected Salary
Permanent 30000	35000
Permanent 50000	55000
Contract 120×250	30000
Contract 100×200	20000

Question 3: Bank Account System using Data Hiding

Program

```

class BankAccount{

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int a,String h,double b){

        accountNumber=a;
    }
}

```

```
        holderName=h;
        balance=b;
    }

    void deposit(double amount){
        balance+=amount;
        System.out.println("Amount Deposited");
    }

    void withdraw(double amount){

        if(amount<=balance){
            balance-=amount;
            System.out.println("Amount Withdrawn");
        }
        else{
            System.out.println("Insufficient Balance");
        }
    }

    void display(){
        System.out.println("Account Number : "+accountNumber);
        System.out.println("Holder Name : "+holderName);
        System.out.println("Balance : "+balance);
    }
}

public class Main{

    public static void main(String args[]){

        BankAccount b=new BankAccount(1001,"Rahul",10000);

        b.deposit(2000);

        b.withdraw(5000);
    }
}
```

```
b.withdraw(9000);
```

```
b.display();
```

```
}
```

```
}
```

Output

Amount Deposited

Amount Withdrawn

Insufficient Balance

Account Number : 1001

Holder Name : Rahul

Balance : 7000.0

Test Cases

Operation	Expected Output
Deposit 2000	Balance increases
Withdraw 5000	Success
Withdraw 9000	Insufficient Balance

Question 4: Student Result Analyzer using Arrays of Objects

Program

```
class Student{
```

```
    int roll;
```

```
    String name;
```

```
    int m1,m2,m3;
```

```
    Student(int r,String n,int a,int b,int c){
```

```
        roll=r;
```

```
        name=n;
```

```
        m1=a;
```

```
        m2=b;
```

```
        m3=c;
```

```

    }

    int total(){
        return m1+m2+m3;
    }

    double average(){
        return total()/3.0;
    }

    String grade(){
        if(average()>=90)
            return "A";
        else if(average()>=75)
            return "B";
        else if(average()>=50)
            return "C";
        else
            return "Fail";
    }
}

public class Main{
    public static void main(String args[]){
        Student s[]={
            new Student(1,"Ajay",90,95,98),
            new Student(2,"Bala",80,82,84),
            new Student(3,"Kumar",70,72,75)
        };
        Student topper=s[0];
        for(Student st:s){
            System.out.println(st.roll+" "+st.name);
            System.out.println("Total="+st.total());
            System.out.println("Average="+st.average());
            System.out.println("Grade="+st.grade());
            if(st.total()>topper.total())
                topper=st;
        }
        System.out.println("\nClass Topper : "+topper.name);
    }
}

```

Output

1 Ajay

Total=283

Average=94.33

Grade=A

2 Bala

Total=246

Average=82.0

Grade=B

3 Kumar

Total=217

Average=72.33

Grade=C

Class Topper : Ajay

Test Cases

Marks	Expected Grade
95,95,90	A
80,82,84	B
70,72,75	C
30,40,35	Fail

Question 5: Shape Area Calculator using Abstract Class & Polymorphism

Program

```
abstract class Shape{  
    abstract double area();  
}  
  
class Circle extends Shape{  
    double r;
```

```

Circle(double r){
    this.r=r;
}
double area(){
    return 3.14*r*r;
}
}

class Rectangle extends Shape{
    double l,b;
    Rectangle(double l,double b){
        this.l=l;
        this.b=b;
    }
    double area(){
        return l*b;
    }
}

class Triangle extends Shape{
    double base,height;
    Triangle(double b,double h){
        base=b;
        height=h;
    }
    double area(){
        return 0.5*base*height;
    }
}

public class Main{
    public static void main(String args[]){
        Shape s[];
        s=new Shape[3];
        s[0]=new Circle(5);
        s[1]=new Rectangle(10,4);
        s[2]=new Triangle(8,6);
        for(Shape sh:s){
            System.out.println("Area = "+sh.area());
        }
    }
}

```

```
}  
}
```

Output

Area = 78.5

Area = 40.0

Area = 24.0

Test Cases

Shape	Input	Expected Area
Circle	r=5	78.5
Rectangle	10,4	40
Triangle	8,6	24